

Agenda:

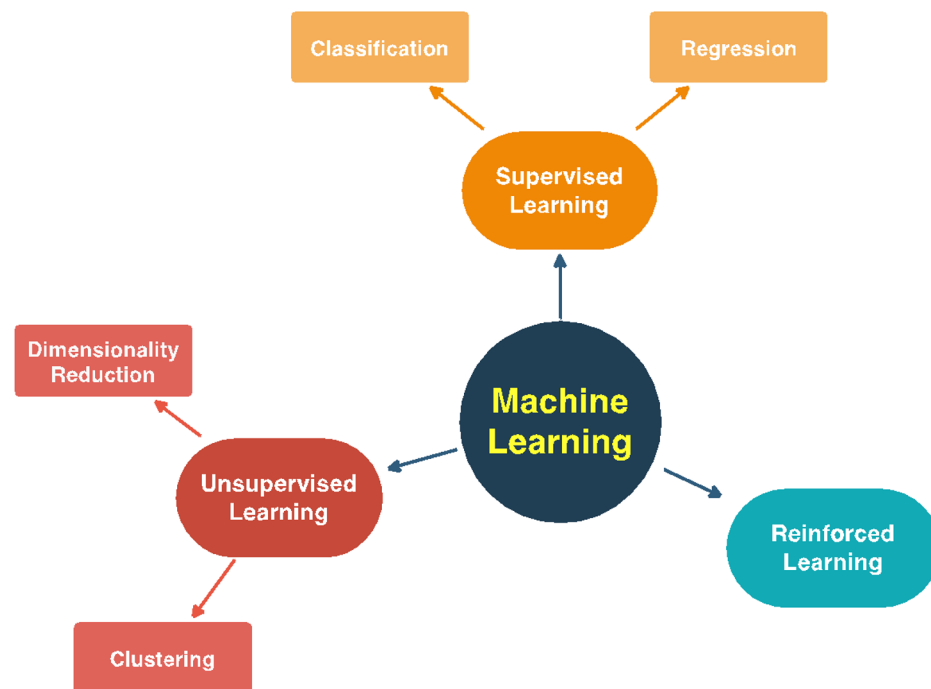
1. Back Propagation
2. Learning rate
3. Optimizers
4. Hands on exercise

Machine Learning subcategories:

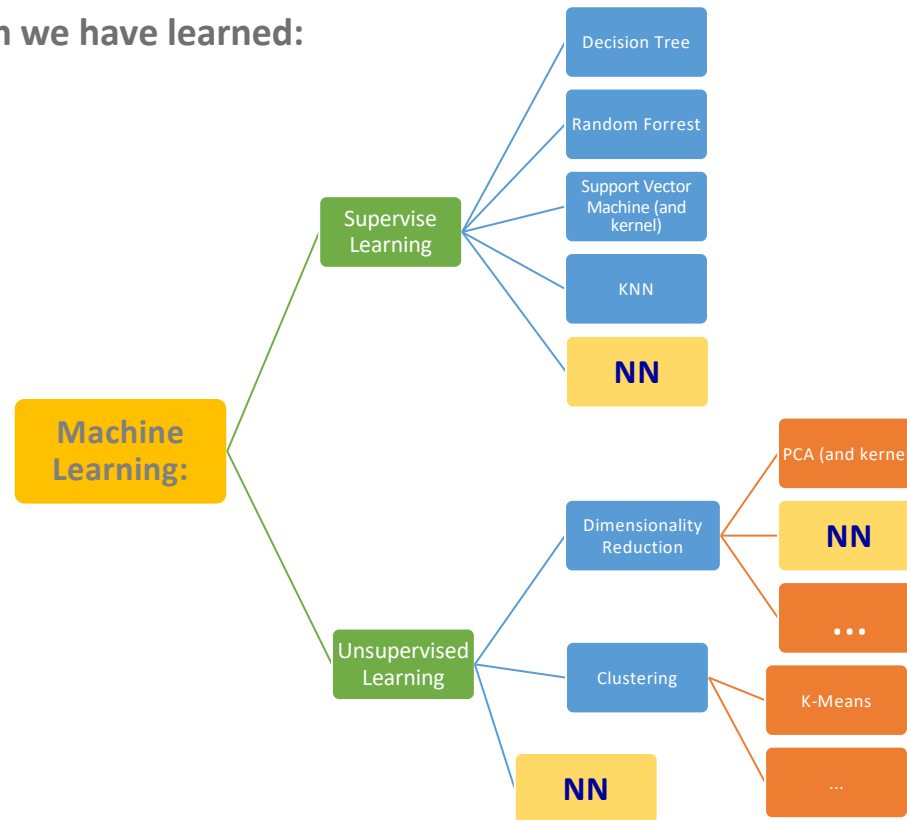
- I. Supervised Learning
- II. Un-supervised Learning
- III. Reinforced learning
- IV. Others...

Where is NN:

Almost all of them



What algorithm we have learned:

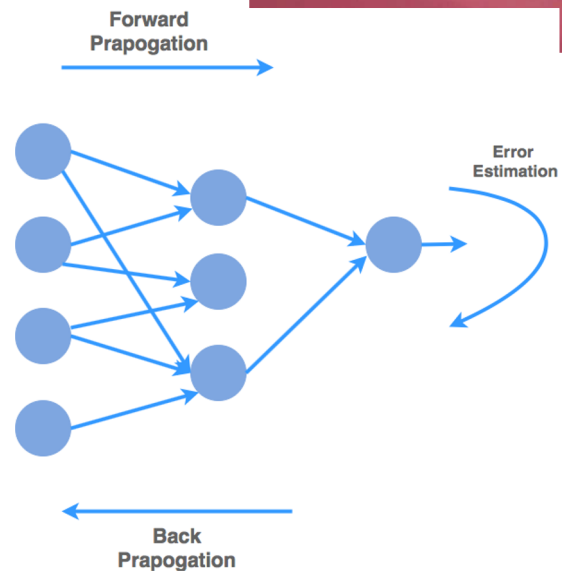


Neural Networks

Intuition behind Neural Network: How it works

Remember the tennis player?

- Forward Propagation (feedforward)
- Backward Propagation



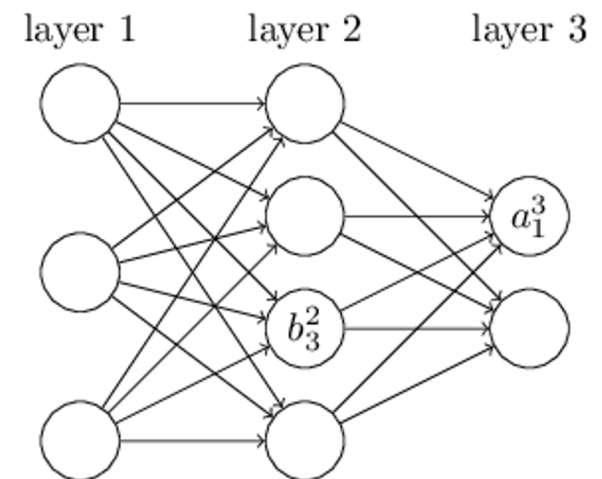
Intuition behind Neural Network: How it works

Notations:

Lets come up with a notation system for activation and weights:

The activation a_1^3 is referring to activation of 1st neuron of the 3rd layer

The weight $w_{1,3}^1$ is the weight connecting the 1st node of first layer to 1st node of the second layer



Intuition behind Neural Network: How it works

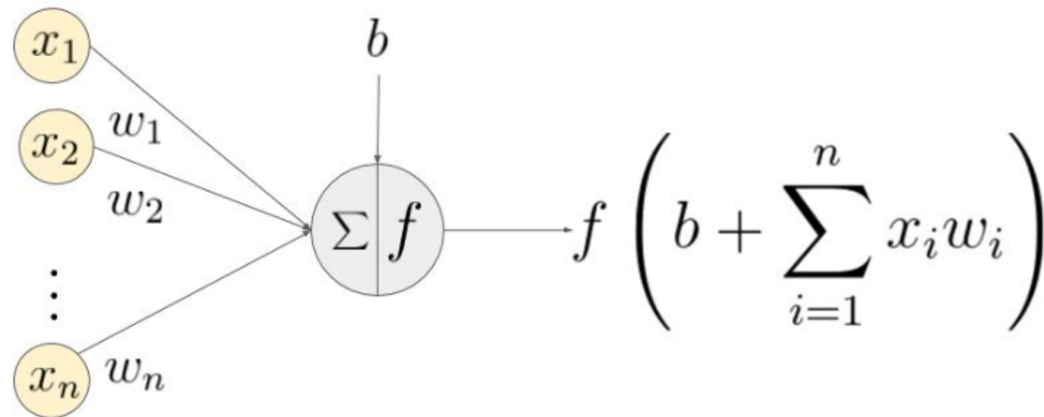
Forward Propagation:

Pre-activation: It is a weighted sum of inputs

$$h_1 = w_1 * x_1 + w_2 * x_2 + \dots + w_n * x_n + b$$

Activation: The Pre-activation is passed to a function

$$z = f(h_1)$$



Intuition behind Neural Network: How it works

Forward Propagation:

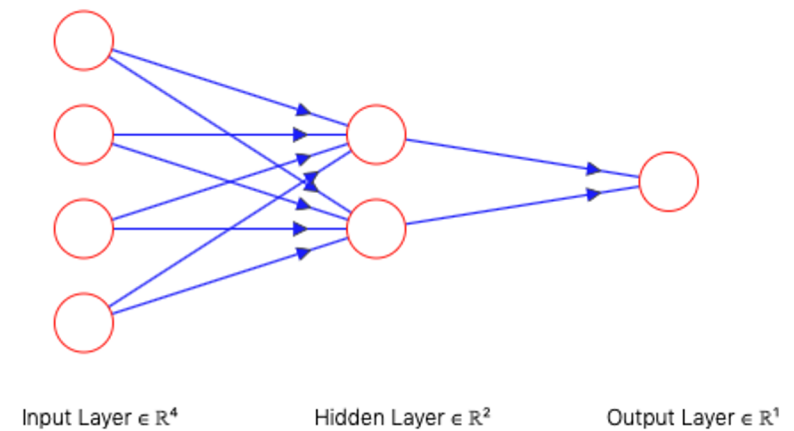
Lets assume a network with:

4 input node

1 hidden layer with 2 nodes

1 output node

(for this example we assume our database has only 1 sample, for now)



Intuition behind Neural Network: How it works

Forward Propagation:

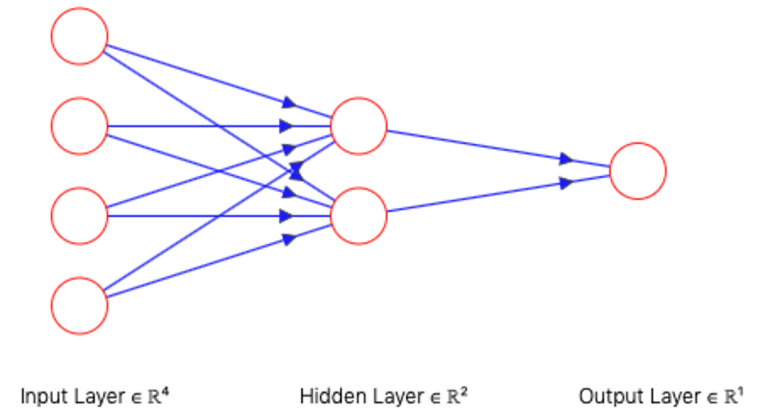
Well first step is to calculate the activation of the nodes through forward prop.

First layer which is the input layer

$$x_i = a_i^{(1)}, i \in 1,2,3,4$$

The weight that connect input to hidden layer can be shown as a matrix:

$$W^{(1)} = \begin{bmatrix} W_{11}^{(1)} & W_{12}^{(1)} & W_{13}^{(1)} & W_{14}^{(1)} \\ W_{21}^{(1)} & W_{22}^{(1)} & W_{23}^{(1)} & W_{24}^{(1)} \end{bmatrix}$$



$$b^{(1)} = \begin{bmatrix} b_1^{(1)} \\ b_2^{(1)} \end{bmatrix}$$

Intuition behind Neural Network: How it works

Forward Propagation:

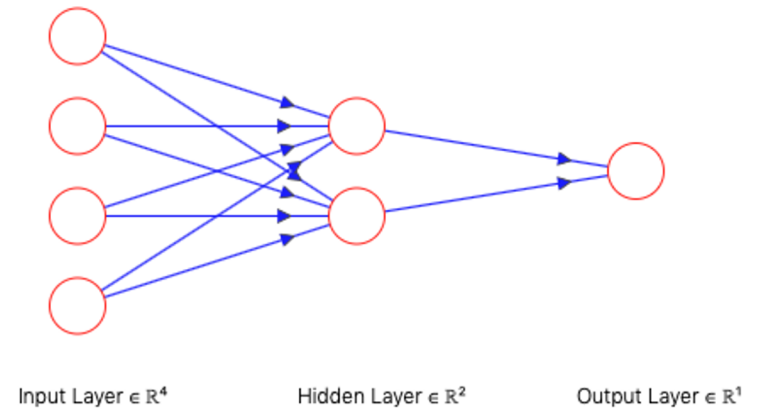
For the hidden and output layer, activation can be calculated,

$$f\left(b + \sum_{i=1}^n x_i w_i\right)$$

as an example (2nd layer):

$$z^{(2)} = W^{(1)}x + b^{(1)}$$

$$a^{(2)} = f(z^{(2)})$$

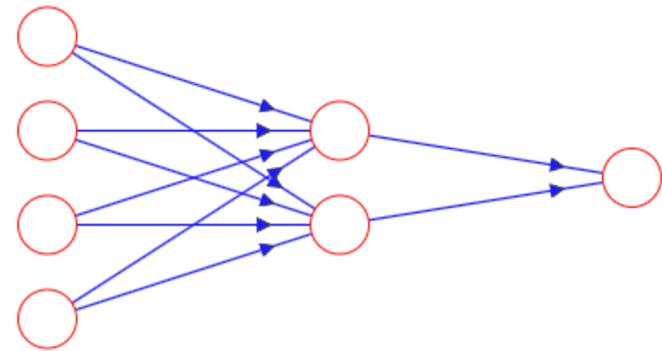


Intuition behind Neural Network: How it works

Forward Propagation:

Note that the forward prop can be written in nice following vectorized format:

$$a^l = f(z)$$
$$z = w^l * a^{l-1} + b^l$$



And in matrix form it will look like (example for 2nd layer)

Input Layer $\in \mathbb{R}^4$

Hidden Layer $\in \mathbb{R}^2$

Output Layer $\in \mathbb{R}^1$

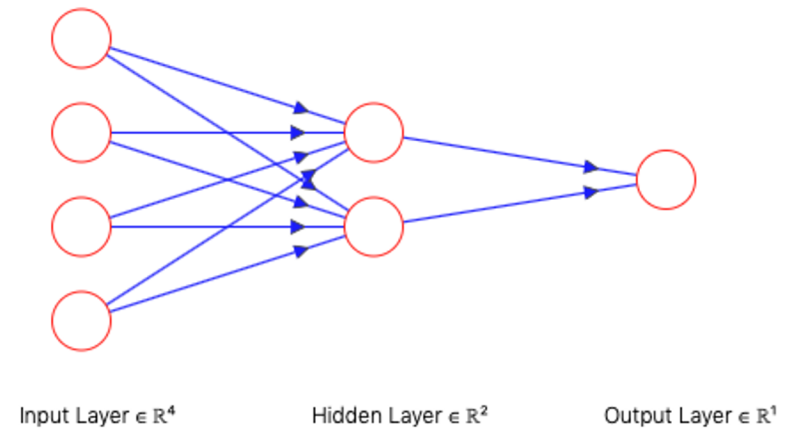
$$z^{(2)} = \begin{bmatrix} W_{11}^{(1)}x_1 + W_{12}^{(1)}x_2 + W_{13}^{(1)}x_3 + W_{14}^{(1)}x_4 \\ W_{21}^{(1)}x_1 + W_{22}^{(1)}x_2 + W_{23}^{(1)}x_3 + W_{24}^{(1)}x_4 \end{bmatrix} + \begin{bmatrix} b_1^{(1)} \\ b_2^{(1)} \end{bmatrix}$$

Intuition behind Neural Network: How it works

Forward Propagation:

When activation in the last layer is found cost can be calculated, ex: MSE

$$C(x) = MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$



Intuition behind Neural Network: How it works

Backward Propagation:

1. After cost is calculated how we can minimize it,
2. What parameters we should change to reduce the cost?
3. The better question is “what parameters I have control over?”

Intuition behind Neural Network: How it works

Backward Propagation:

In order to train our model we have to repeatedly adjust the weights and biases of the neural network to improve the model and decrease the cost (i.e. reduce the differences between the predicted values and the real target values)

How much should I adjust the weights, that is determined by the **gradient of cost** with respect to each parameter.

Cost is a function of what?

Weights and Biases

Intuition behind Neural Network: How it works

Backward Propagation:

Consider this function

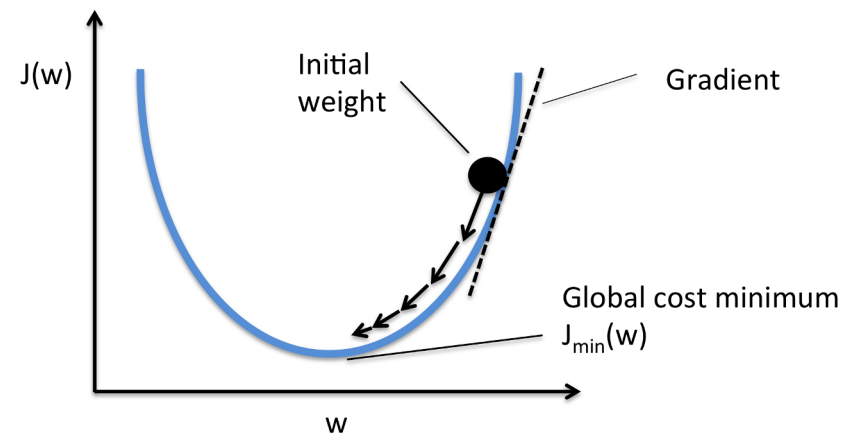
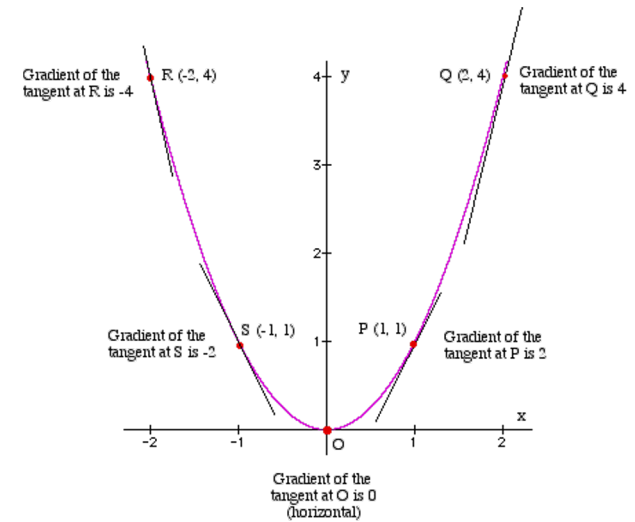
$$C(w) = (w_1 + w_2 + \dots + w_n)$$

Gradient of function C is a vector of Partial derivatives

$$\frac{\partial C}{\partial w} = \left[\frac{\partial C}{\partial w_1}, \frac{\partial C}{\partial w_2}, \dots, \frac{\partial C}{\partial w_n} \right]$$

Our job is to find these partial derivatives and train the model based on gradients.

Remember we are after finding out how much change in weights affect the total error



Intuition behind Neural Network: How it works

Backward Propagation:

How to find gradient of cost with respect to a specific weight or bias?

Forward prop: climb the ladder

Backward prop: go down the ladder

Chain Rule

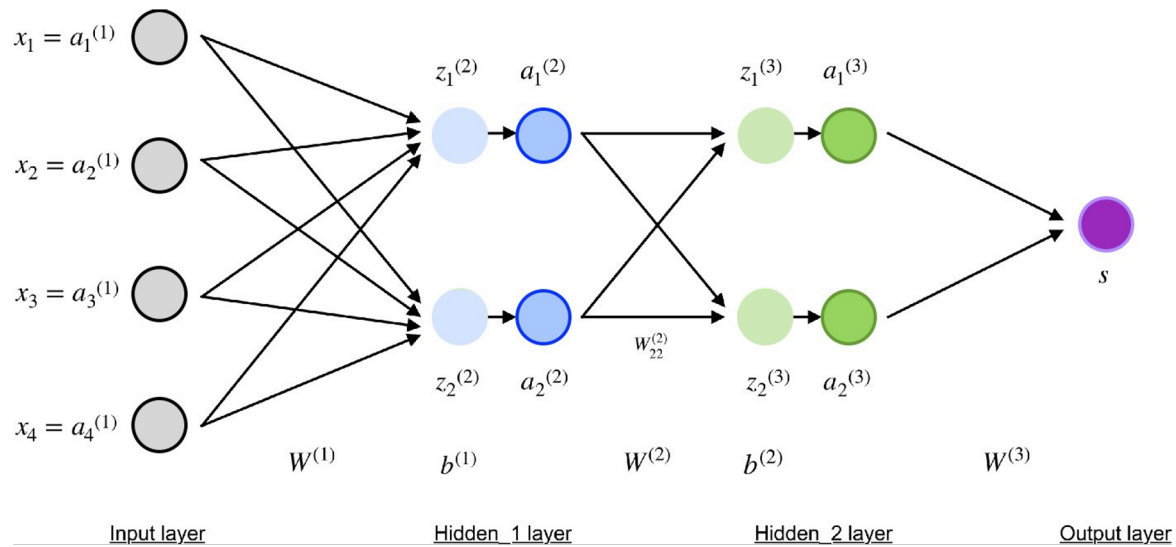
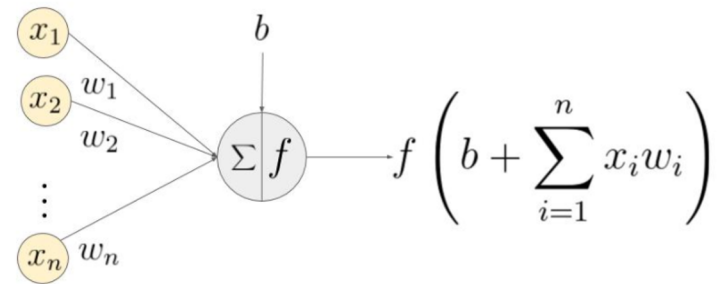
$$C = f(x), x = f(y), y = f(u)$$

$$\frac{\partial C}{\partial u} = \frac{\partial C}{\partial x} * \frac{\partial x}{\partial y} * \frac{\partial y}{\partial u}$$

Intuition behind Neural Network: How it works

Backward Propagation:

What we have done in forward prop (climb the ladder):



Intuition behind Neural Network: How it works

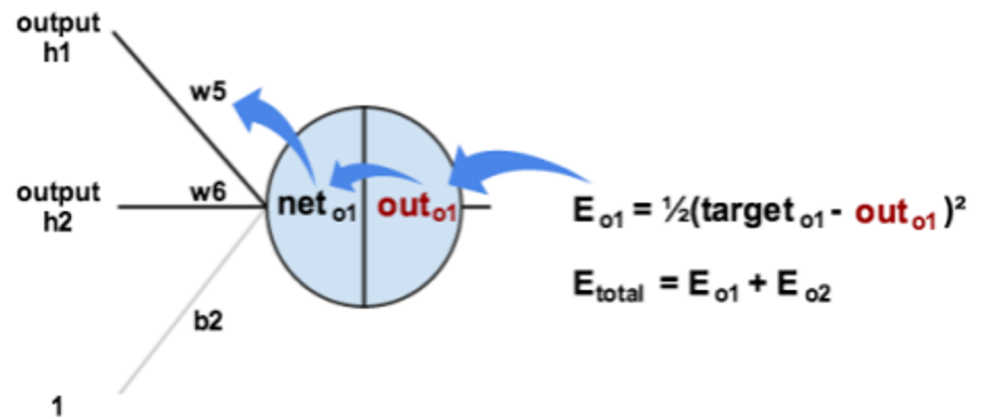
Backward Propagation:

What we have done in forward prop (climb the ladder)

Note that E in the picture is the same as cost C

$$\frac{\partial C}{\partial w_5} = \frac{\partial C}{\partial out_{o1}} * \frac{\partial out_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial w_5}$$

What are each of these terms?



Intuition behind Neural Network: How it works

Backward Propagation:

Note that E in the picture is the same as cost C

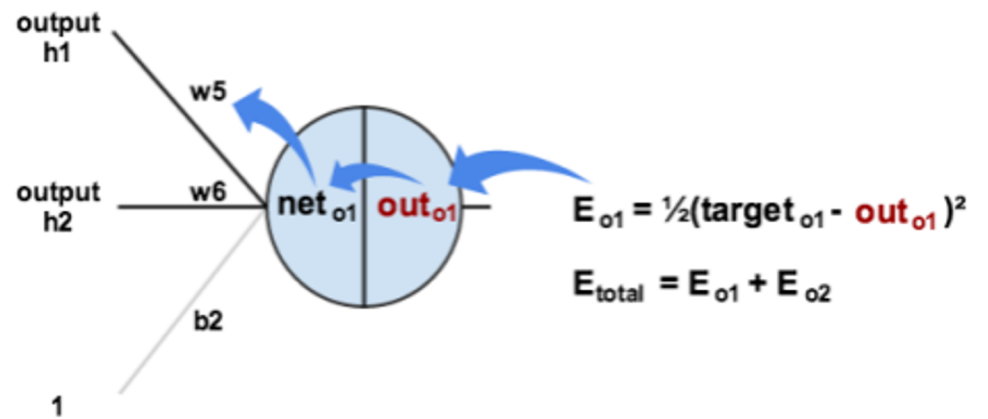
$$\frac{\partial C}{\partial w_5} = \frac{\partial C}{\partial out_{o1}} * \frac{\partial out_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial w_5}$$

What are each of these terms?

$\frac{\partial C}{\partial out_{o1}}$ is the derivative of Cost Function

$\frac{\partial out_{o1}}{\partial net_{o1}}$ is the derivative of activation function

$\frac{\partial net_{o1}}{\partial w_5}$ is the values of nodes at previous layer



Intuition behind Neural Network: How it works

Backward Propagation:

Finally when gradients are found weights and biases can be updated (adjusted) using Gradient Descent as follow:

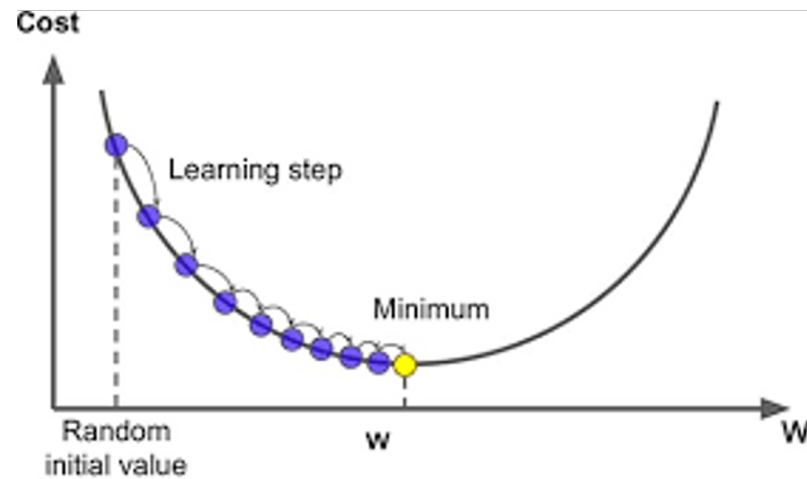
η is the learning rate

$$\theta_{n+1} = \theta_n + \eta \nabla(C(x_n))$$

$$w_{n+1} = w_n + \eta \frac{\partial C}{\partial w}$$

Same for biases

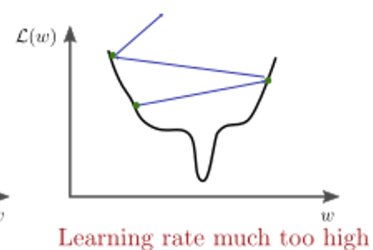
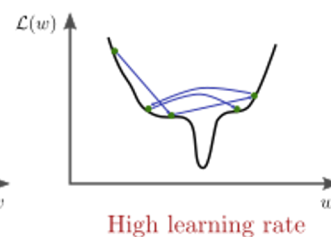
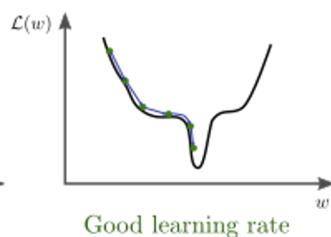
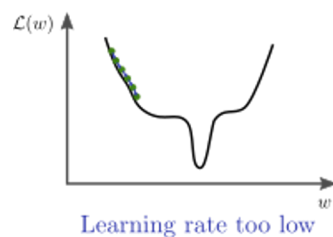
$$b_{n+1} = b_n + \eta \frac{\partial C}{\partial b}$$



More on Learning Rate:

$$w_{n+1} = w_n + \eta \frac{\partial \mathcal{C}}{\partial w}$$

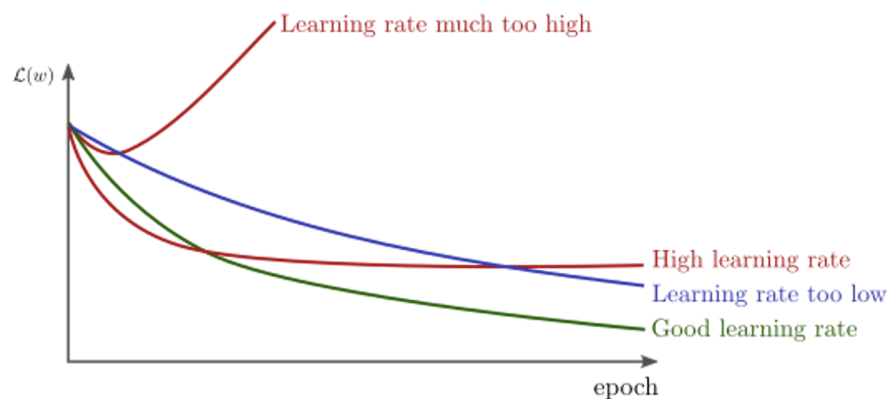
η is the learning rate.



Adaptive Learning Rate:

$$\eta = \eta_{initial} + \frac{1}{(1 + decay * iter)}$$

Decay is another Hyper-parameter



Hyper-Parameters and other unknowns:

In NN there are several parameters that determine the architecture of Neural network and also dictate how a network should train. These parameters are very problem-dependent and need to be chosen wisely. When you use a library like Keras all of these parameters have a default value but you should be aware of it. Here are a list of unknown parameters and properties that you should choose for your network:

- Number of layers and nodes
- Learning Rate
- Decay rate
- Momentum
- Dropout
- Weight initialization
- Epoch and batch size
- Activations

Intuition behind Neural Network: How it works

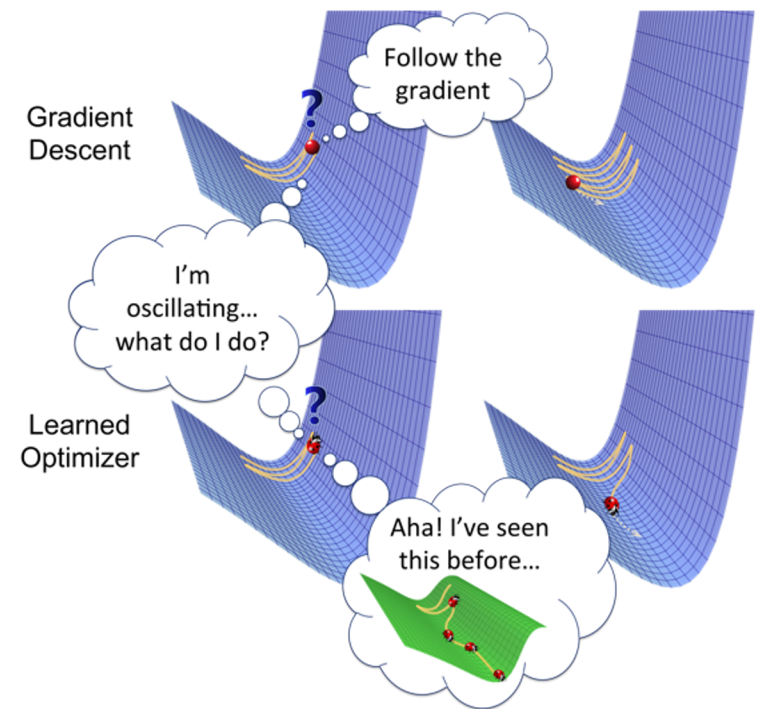
Optimization:

"The action of making the best or most effective use of a situation or resource." from google dictionary

"An optimization problem is the problem of finding the best solution from all feasible solutions." Wikipedia

How we change NN parameters to minimize the cost is an Optimization problem.

The optimizer will determine how fast you get to the optimal point (where the cost is minimum)



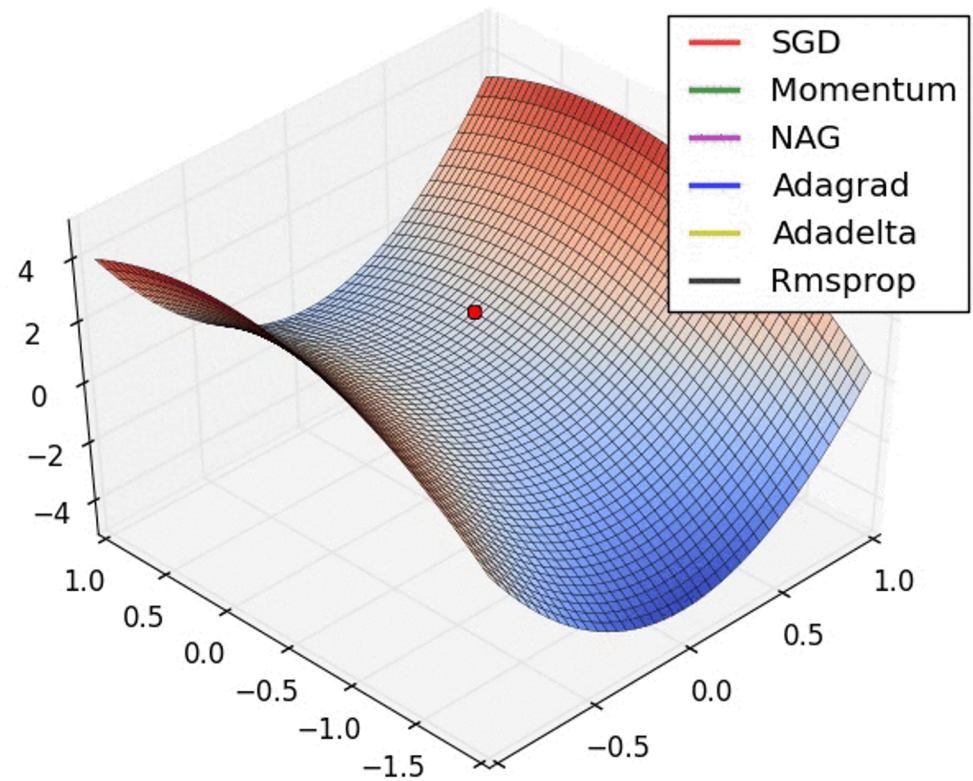
Intuition behind Neural Network: How it works

Optimization:

Different type of Optimizers:

1. Stochastic Gradient decent
2. Using Momentum
3. Nesterov Accelerated Gradient Descent (NAG)
4. Adaptive Gradient Algorithm (Adagrad)
5. Root Mean Square Propagation (RMSProp)
6. Adam (Adaptive moment estimation)

Adam Optimizer is an update to RMSprop, mixed with Adagrad



Intuition behind Neural Network: How it works

Optimization:

Adam equations:

η , is the learning rate

β_1 , is one constant (usually 0.9)

β_2 , is one constant (usually 0.999)

ϵ , usually $1e-8$

<http://alexlenail.me/NN-SVG/index.html>

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$
$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

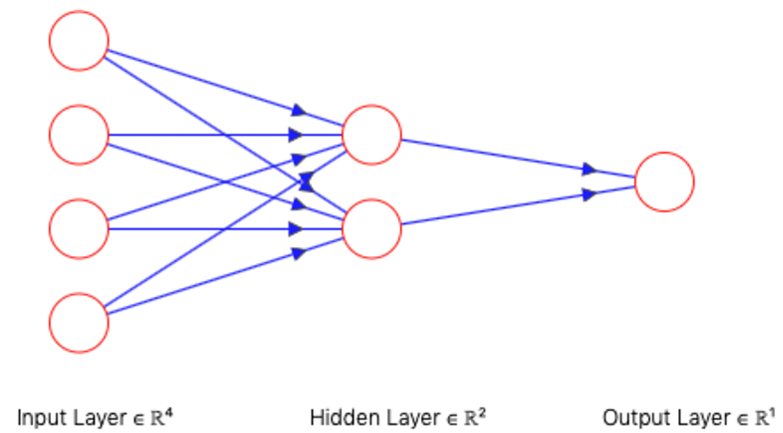
$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

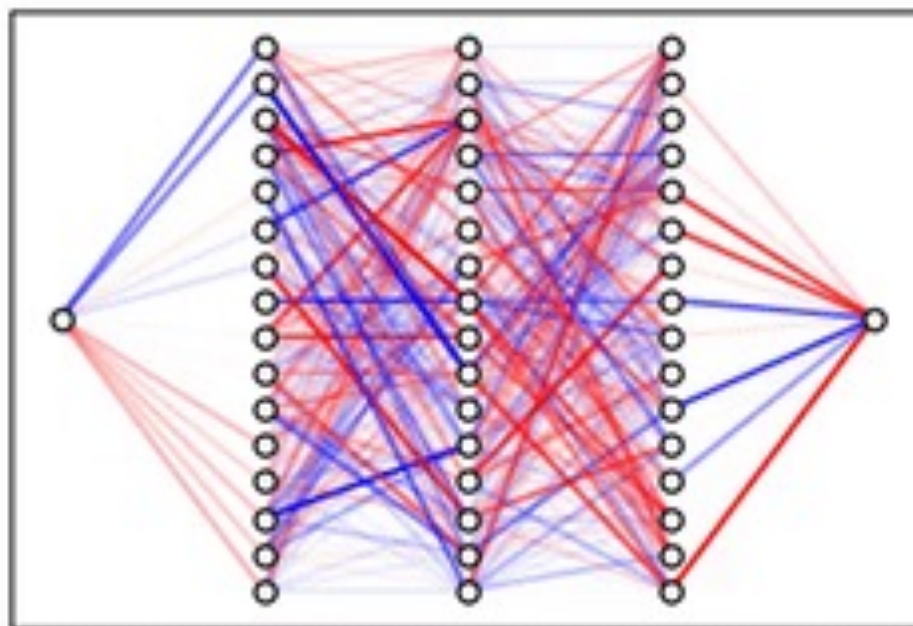
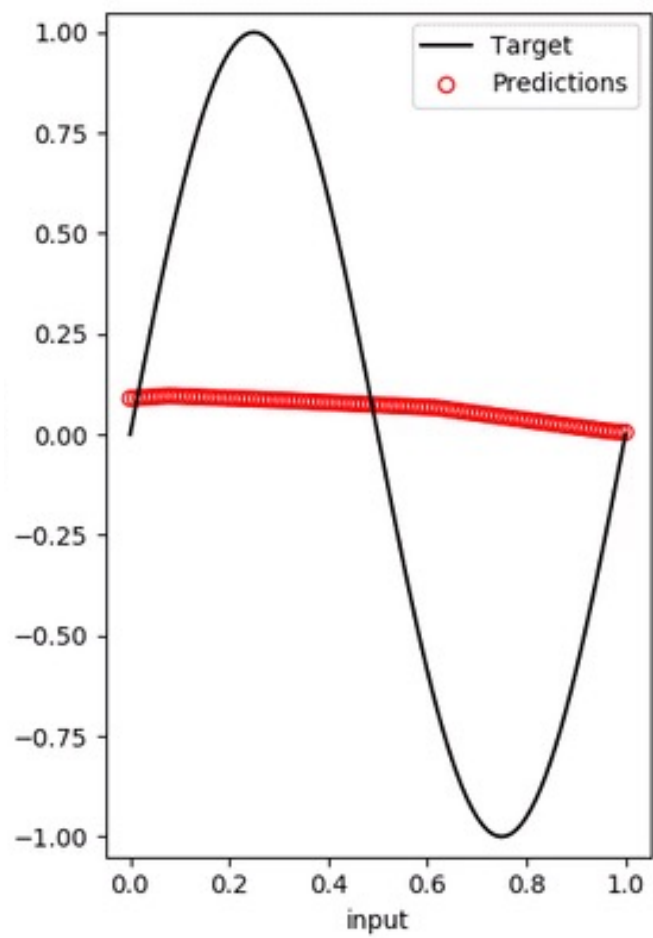
$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$w_t = w_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

A good website to draw NN architectures:

<http://alexlenail.me/NN-SVG/index.html>







Python:

Hands on exercise:

Lets discover the effect of Adam optimizer

Learning rate,

Activation functions

Optimizers: <https://keras.io/api/optimizers/>